

Inference Engineering Assignment

Part A – Theory

September 9, 2026

Q1. What’s the difference between latency and throughput in model inference, which factor will influence model’s performance more and how?

Latency = how much time it takes to finish ONE single request. Like, you ask a question, and the time it takes to get the answer back — that’s latency.

Throughput = how much total work the system can do in a given time. Like how many requests or how many words (tokens) it can process per second, for everyone together.

Which one matters more? It depends:

If it’s a chatbot (like people are waiting for a reply right now), latency matters more. People don’t like waiting.

If it’s something like processing thousands of files overnight (nobody is watching it live), throughput matters more. You just want the whole job done fast, doesn’t matter if one single item takes a bit longer.

Important trade-off: If you increase the batch size (processing many requests together), throughput goes up (system works more efficiently), BUT each individual request now has to wait a little longer for its turn — so latency goes up a bit too. You can’t get the best of both at the same time.

Extra info:

- In LLMs, latency has two parts:
 - TTFT (Time To First Token) — how long you wait before you see the FIRST word of the answer.
 - TPOT (Time Per Output Token) — how fast each next word comes after that.
- Throughput is usually measured in tokens/second, counting ALL users together, not just one person.
- In real companies, they set a target like “95% of replies should come within X seconds” and then try to push throughput as high as possible without breaking that rule.

Q2. Explain what KV cache is and how it helps with model's generation?

When a model writes text, it does it one word (token) at a time. But to predict the next word, it needs to “look back” at all the previous words using something called K and V vectors (part of the attention mechanism).

Without a cache: the model would have to recalculate K and V for ALL previous words, again and again, every single time it generates a new word. Waste of time and power.

With KV cache: we just save (store) the K and V values we already calculated. Next time, we only calculate K and V for the NEW word, and reuse the old saved ones. Much faster!

Trade-off: This cache needs memory (RAM/VRAM) to store all the saved values. So the longer the conversation, or the more people using it at once, the more memory it eats up.

Extra info:

- The size of the KV cache depends on: sequence length $\times$ number of layers $\times$ number of heads $\times$ head size. That's why very long conversations cause memory problems.
- MQA and GQA are tricks that make multiple attention heads share the same K/V, so less memory is used.
- vLLM uses something called PagedAttention — it manages memory kind of like how a computer's OS manages RAM in pages. This avoids wasting memory and lets more people use the model at the same time.

Q3. What is quantization, and what's the trade-off between int8/int4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed.

Quantization means taking a model's numbers (weights), which are normally stored in high precision (like FP16, meaning lots of decimal accuracy), and converting them into lower precision numbers (like INT8 or INT4, which use fewer bits).

Why do this? Because smaller numbers = smaller model size = faster processing (computers do integer math faster/cheaper than floating point math).

Trade-off: Since we're throwing away some precision, the model can become a little less accurate.

- INT8 $\rightarrow$ small accuracy loss (not too bad)
- INT4 $\rightarrow$ bigger accuracy loss (because we're squeezing values into a much smaller range)

Formula to remember:

Quantize: $x_q = \text{round}(x/S) + Z$

Dequantize: $x = S \times (x_q - Z)$

Where:

- x = the original number
- S = scale (a fixed number we choose)
- Z = zero-point (usually 0)

- x_q = the new, smaller quantized number

Example: weight $x = 0.72$ (FP16), scale $S = 0.0078$, zero-point $Z = 0$

$$x_q = \text{round}(x/S)$$

$$x_q = \text{round}(0.72/0.0078) = 92$$

So this weight will be stored as 92 in INT8. If we want it back in float form, we dequantize it:

$$x = S \times (x_q - Z) = 0.0078 \times 92 \approx 0.718.$$

Q4. Name one real inference serving framework (like vLLM or TensorRT-LLM) and briefly describe one optimization it does.

Example: vLLM — a tool used to run and serve LLMs efficiently.

Optimization 1: PagedAttention

This manages GPU memory smartly during text generation (similar to how computer OS manages memory in “pages”). It reduces wasted memory, so the system can handle more requests together, which makes everything faster overall.

Optimization 2: Continuous Batching

Instead of waiting for a full batch of requests to be ready before processing, vLLM keeps adding new requests as they come in and processes them together on the fly. This keeps the GPU always busy, which increases speed and throughput.

Because of these tricks, vLLM is great for serving many users at the same time without slowing down.

Extra info:

- TensorRT-LLM (by NVIDIA) is another popular framework. It uses tricks like kernel fusion (combining multiple small operations into one to save time) and supports different quantization formats to run faster on NVIDIA GPUs.
- vLLM also supports tensor parallelism (splitting the model across multiple GPUs) and various quantization methods — not just PagedAttention and continuous batching.